

CTLE: Compressed-Domain TinyLM Inference on an ESP32-P4 Microcontroller

Emilio Flores, *Graduate Student Member, IEEE*

Abstract—Tiny language models can fit the application class of Internet-of-Things nodes, but their weights still exceed the memory and bandwidth envelope of microcontrollers. This letter presents the Compressed Tensor–LUT Engine (CTLE), a compressed-domain execution engine for transformer inference on ESP32-P4-class microcontrollers. CTLE stores each large weight matrix as a 16-entry learned codebook plus packed 4-bit indices, and fuses nibble decoding, LUT access, and multiply-accumulate inside the matrix–vector kernel, avoiding dense weight reconstruction and scale-based dequantization in RAM. On TinyStories-15M, CTLE-PSO compresses model weights from 60.77 MB to 7.61 MB (7.98×) and achieves 0.965 tok/s on the ESP32-P4 Nano with 21.5 MB PSRAM free. A multi-format firmware comparison shows that INT4 block-wise quantization improves quality but incurs a 31–34 % throughput penalty due to per-group scale reads from PSRAM. CTLE-5b ($K=32$, 5-bit indices, 9.51 MB) surpasses INT4BW in both quality (PPL 10.00 vs. 10.71 on TinyStories) and throughput (1,014 vs. 1,356 ms/tok) by eliminating per-group scale reads. P-CTLE32 upgrades activation quantization to 32 levels, halving quality loss (PPL 33.14→20.30) with no measurable throughput penalty.

Index Terms—TinyML, embedded inference, model compression, microcontrollers, TinyLM.

I. INTRODUCTION

Low-power embedded systems increasingly need local language understanding or generation, but even “tiny” transformer models stress microcontroller memory. The bottleneck is not only model size: conventional low-bit formats often require scale tables, dequantization paths, or dense temporary tensors before a matrix–vector product can be executed. On devices such as the ESP32-P4, with external Flash and PSRAM but limited on-chip SRAM and memory bandwidth, this runtime expansion is as important as the nominal bit-width.

This letter introduces CTLE, a compressed-domain execution engine for TinyLM inference. CTLE executes directly from the compressed representation by streaming packed 4-bit indices, looking up a small learned codebook, and accumulating in the hot loop—without materializing dequantized weight matrices in SRAM or PSRAM.

The contributions are: (1) CTLE, a streaming learned-codebook execution engine with a fused C runtime for ESP32-P4, serving as the common infrastructure for all design points; (2) codebook optimization with K-means, GA, and PSO; (3) a multi-format firmware comparison against INT4 baselines on the same hardware; and (4) two design points enabled by the same engine: **CTLE-5b** ($K=32$, 5-bit), which matches

E. Flores is with the Escuela de Ingeniería Informática, Universidad de Valparaíso, Valparaíso, Chile, and with the Faculty of Engineering and Business, Universidad de las Américas, Santiago, Chile (e-mail: emilio.flores@postgrado.uv.cl). This work was supported by ANID BECAS/DOCTORADO NACIONAL 21242003.

TABLE I
RUNTIME ASSUMPTIONS FOR REPRESENTATIVE QUANTIZATION METHODS.

Method	Target	Metadata	Runtime assumption
INT4U	MCU	one scale	scalar dequantization
INT4BW	MCU/GPU	group scales	scale loads in hot loop
GPTQ/AWQ	GPU	calibration	fused GPU kernels
LLM.int8()	GPU	outliers	mixed 8/16-bit GEMM
CTLE/CTLE-5b	MCU	K -value LUT	compressed stream matvec
P-CTLE	MCU/NPU	product LUT	lookup accumulation

INT4BW quality and exceeds its throughput without PSRAM scale traffic, and **P-CTLE32** (W4/A5), which demonstrates a multiplier-free product-LUT datapath at 1.201 tok/s.

II. RELATED WORK

MCU inference work emphasizes memory, bandwidth, and energy as primary bottlenecks [1]–[3]. LUT-based methods have targeted activation nonlinearities [4], [5], binarized networks [6], and low-bit matvec execution [14], but not the streaming learned-codebook paradigm used here. CTLE differs from prior LUT work in three ways: (i) the codebook is *learned per tensor* via K-means or metaheuristic optimization, not fixed quantization levels; (ii) the LUT is 16 entries (64 B) per matrix, small enough to remain register/cache-resident; and (iii) it is evaluated end-to-end on a real RISC-V microcontroller with a full transformer forward pass.

GPU-oriented LLM quantizers such as GPTQ, SqueezeLLM, LLM.int8(), and AWQ [7]–[10] achieve strong quality on large models, but they assume GPU-resident tensors and fused GPU kernels. Table I summarizes the runtime assumptions that distinguish these classes: CTLE’s differentiator is a learned non-uniform 4-bit representation executed directly from a compressed stream on an MCU.

III. COMPRESSED-DOMAIN EXECUTION

For a matrix $W \in \mathbb{R}^{M \times N}$, CTLE stores a codebook $L \in \mathbb{R}^{16}$ and an index matrix $I \in \{0, \dots, 15\}^{M \times N}$. Two indices are packed per byte. Inference computes

$$y_m = \sum_{n=0}^{N-1} L[I_{m,n}] x_n, \quad (1)$$

streaming packed bytes and decoding low/high nibbles in row-major order. The 16-entry LUT is 64 B, small enough to remain register- or cache-resident during the row loop. As shown in Fig. 1, no dense copy of W is reconstructed in SRAM or PSRAM.

The binary format contains a 40-byte model header followed by tensor blocks tagged as FP32 (0), CTLE (1), INT4U (2),

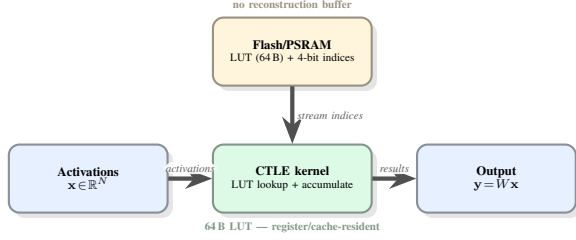


Fig. 1. CTLE architecture. Packed 4-bit indices stream from Flash into the matvec kernel; no dense dequantized weight matrix is materialized.

Transient overhead per weight matrix W (this work)

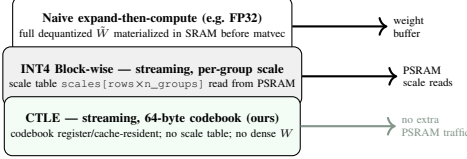


Fig. 2. Memory-overhead comparison for three deployment paths evaluated in this work. Naive expand-then-compute materializes a full weight matrix in SRAM; INT4BW streams nibbles but reads per-group scales from PSRAM in the hot loop; CTLE streams nibbles against a 64-byte register/cache-resident codebook, adding no extra PSRAM traffic beyond the index stream (Fig. 1).

INT4BW (3), P-CTLE (4), or CTLE-5b (5). A single tag-switch selects the matvec kernel; all formats share the same transformer forward-pass code, ensuring a fair hardware comparison. Fig. 2 contrasts the transient-RAM cost of each class.

a) *INT4 baselines*.: INT4U stores one symmetric scale per tensor. It is fast but collapses on the 9.2M-entry token embedding whose weights span multiple orders of magnitude. INT4BW improves quality with one scale per 32-column group, at the cost of additional PSRAM reads in the hot loop: each row of a 288-column matrix requires nine group-scale loads before the nibble stream can be accumulated. CTLE instead reuses 16 centroids across all rows of the tensor, keeping the access pattern close to a pure streaming kernel. This has a measurable latency consequence: INT4BW is 31 % slower than CTLE-PSO and 34 % slower than CTLE-5b (Section V), a gap that arises not from the bit-width but from the second PSRAM access stream introduced by group scales.

The firmware dispatch path is a single tag-switch at the start of each matvec. INT4U multiplies each quantized index by a single per-tensor scale; INT4BW multiplies by a per-group scale that must be loaded from PSRAM for each row-group. CTLE eliminates this scale-based dequantization step: centroids are stored as FP32 in the codebook, so the inner loop executes $\text{acc} += L_w[w_{\text{idx}}] \cdot x_n$ directly—a standard multiply-accumulate, but without any preceding scale conversion or dense weight reconstruction. P-CTLE goes further by additionally quantizing the activation vector x to 16 uniform levels and precomputing a 256-entry product LUT $L_p[a, w] = L_a[a] \cdot L_w[w]$ once per matvec call. The inner loop then becomes $\text{acc} += L_p[a_{\text{idx}} \| w_{\text{idx}}]$, removing the per-element float multiply entirely. P-CTLE32 extends this to 32 activation levels (512-entry LUT, index $a \cdot 16 + w$); the doubled LUT build (512 muls) is amortized over 32,000 vocabulary

Algorithm 1 CTLE fused Tensor-LUT kernel (N even).

Require: activation $x \in \mathbb{R}^N$, packed byte array P , codebook $L \in \mathbb{R}^{16}$

Ensure: output $y \in \mathbb{R}^M$

```

1: for  $m = 0$  to  $M - 1$  do
2:    $\text{acc} \leftarrow 0$ 
3:   for  $j = 0$  to  $N/2 - 1$  do
4:      $B \leftarrow P[m, j]$ 
5:      $\text{acc} += x_{2j} \cdot L[B \& 0x0F] + x_{2j+1} \cdot L[B \gg 4]$ 
6:   end for
7:    $y_m \leftarrow \text{acc}$ 
8: end for
```

TABLE II

PER-TENSOR RELMSE. PSO IMPROVES THE LARGE EMBEDDING; SMALL PROJECTIONS ARE ALREADY NEAR THE K-MEANS OPTIMUM.

Tensor	K-means	GA	PSO
tok_emb (9.2M)	0.01190	0.01191	0.01013
wq.0 (82K)	0.01553	0.01553	0.01564
wk.0 (82K)	0.01473	0.01473	0.01474
wl.0 (83K)	0.00997	0.00997	0.00999
Mean (all tensors)	0.01264	0.01264	0.01265

steps and does not affect throughput.

A 5-bit extension, CTLE-5b, uses $K = 32$ centroids and packs 8 indices per 5-byte group. Decoding loads 5 bytes into a 64-bit register and extracts eight 5-bit fields via right-shifts and masks (decode5_group), with no per-group PSRAM scale reads. Both extensions share the same firmware tag-switch and forward-pass infrastructure.

IV. CODEBOOK OPTIMIZATION

The baseline codebook is obtained by 1-D K-means on each flattened tensor. K-means is fast and near-optimal on small projection matrices, but large embedding tensors have heavier-tailed distributions and higher local-minimum risk. We therefore warm-start GA and PSO from K-means and optimize reconstruction MSE over a random subsample of up to 50k weights. The assignment and MSE kernels are compiled with Numba JIT, keeping full-model compression within minutes on a laptop CPU (no GPU required).

GA represents each individual as a K -value codebook ($K = 16$ for CTLE, $K = 32$ for CTLE-5b) and uses tournament selection, BLX- α crossover, Gaussian mutation, and elitism. PSO uses 60 particles, inertia decay from 0.9 to 0.4, and K-means perturbations as initial positions. Both optimizers run offline; for a fixed bit-width the deployed runtime is unchanged—only the centroid values and packed-index assignments differ. PSO does not increase Flash footprint, PSRAM allocation, or runtime complexity.

Table II shows that PSO’s gain concentrates where it matters: the 9.2M-entry token embedding (tok_emb), the largest tensor by two orders of magnitude. For small projection matrices ($\leq 82K$ weights), K-means is already near-optimal and GA/PSO offer no improvement. Expensive metaheuristic search can therefore be reserved for high-impact tensors.

TABLE III
COMPRESSION AND QUALITY ON TINYSTORIES-15M. “COMP. S” IS
OFFLINE COMPRESSION TIME; RUNTIME PRODUCT-LUT REBUILDS ARE
EXCLUDED.

Method	Bits	MB	PPL TS	PPL WT2	Comp. s
FP32	32	60.77	9.35	6,127	–
INT4U	4	7.61	11,070	533,069	<1
INT4BW	~5	9.51	10.71	7,120	<1
CTLE-KM	4	7.61	19.23	19,239	73.6
CTLE-GA	4	7.61	18.98	19,114	207.7
CTLE-PSO	4	7.61	15.93	16,827	198.3
P-CTLE	4	7.61	33.14	29,049	181.4
P-CTLE32	W4/A5	7.61	20.30	19,095	181.4
CTLE-5b	5	9.51	10.00	8,226	374

V. EXPERIMENTAL RESULTS

a) *Setup.*: Model: TinyStories-15M (Llama-2, $d = 288$, 6 layers, 32k vocab) [11]. Hardware: ESP32-P4 Nano, RISC-V HP core at 360 MHz, 32 MB QSPI PSRAM, 16 MB Flash. Quality is evaluated on a 192-token TinyStories (TS) probe [12] and on the WikiText-2 (WT2) test split [13] using non-overlapping 256-token windows. WT2 absolute PPL is high because TinyStories-15M was not trained on Wikipedia; WT2 is used here to verify relative degradation ordering, not as a cross-paper leaderboard against OPT or LLaMA models. Compression runs on an Apple M-series laptop (no GPU); timing covers the full model.

b) *Quality ablation.*: Table III summarizes all evaluated formats. INT4U collapses (PPL = 11,070/533,069) because one global scale cannot represent the token embedding’s range. INT4BW recovers quality (10.71/7,120) at 25 % higher footprint (9.51 vs. 7.61 MB) and with per-group scale overhead. CTLE-KM achieves 19.23/19,239 in 73.6 s; CTLE-PSO reaches 15.93/16,827 (−17.2 %/−12.5 % vs. K-means on TS/WT2) in 198.3 s. The consistent ranking across both corpora confirms that PSO finds a genuinely better codebook and not a corpus-specific optimum. **CTLE-5b** ($K = 32$, 5-bit, 374 s) reaches PPL 10.00/8,226, surpassing INT4BW at the same 9.51 MB footprint. **P-CTLE32** (W4/A5) halves P-CTLE’s quality penalty (33.14→20.30) with no additional compression cost.

c) *Hardware throughput.*: Table IV reports averages over three prompts (4, 8, and 6 prefill tokens; 64 tokens generated each), using seed 0xCAFEBABE, temperature 0.8, top- p 0.9. CTLE-PSO matches INT4U throughput (1,035.9 vs. 1,040.6 ms/tok) because both read a single metadata value before streaming nibbles, while INT4BW pays a 34 % penalty (1,355.8 ms/tok) from nine per-row PSRAM scale loads. Prefill ms/tok tracks generation ms/tok within 2 % for all methods (Table V), confirming that the cost is dominated by a single transformer forward pass regardless of sequence position.

CTLE-5b ($K=32$, 5-bit indices, 9.51 MB) achieves 1,014.2 ms/tok (0.986 tok/s)—34 % faster than INT4BW at identical footprint—because `decode5_group` loads 5 bytes into a 64-bit register and extracts eight 5-bit indices via shifts, adding no PSRAM scale reads. Its TinyStories PPL (10.00) surpasses INT4BW (10.71). **P-CTLE32** raises activation levels from 16 to 32 (512-entry product LUT); TinyStories

TABLE IV
ESP32-P4 BENCHMARK AVERAGED OVER THREE PROMPTS (64 TOKENS
GENERATED).

Method	Model KB	Free PSRAM	ms/tok	tok/s
INT4U	7,431	21,566	1,040.6 ± 0.6	0.961
INT4BW	9,285	19,682	1,355.8 ± 0.4	0.737
CTLE-PSO	7,433	21,566	1,035.9 ± 1.1	0.965
P-CTLE	7,433	21,566	832.5 ± 0.8	1.201
CTLE-5b	9,290	19,798	1,014.2 ± 0.7	0.986
P-CTLE32	7,433	21,566	832.4 ± 1.5	1.201

TABLE V
PER-PROMPT PREFILL AND GENERATION LATENCY CONFIRMING
FORWARD-PASS DOMINANCE (CTLE AND P-CTLE SHOWN; CTLE-5B
AND P-CTLE32 AVERAGES APPEAR IN TABLE IV).

Method	Prompt (tok)	Prefill ms/tok	Gen ms/tok
CTLE	once_upon (4)	1032.4	1034.7
CTLE	tom_dog (8)	1023.4	1035.9
CTLE	lily (6)	1014.9	1037.0
P-CTLE	once_upon (4)	830.0	831.3
P-CTLE	tom_dog (8)	808.8	834.2
P-CTLE	lily (6)	812.6	832.0

PPL drops from 33.14 to 20.30 with no throughput change (832.4 ms/tok), as the 512-mul LUT build is amortized over 32,000 vocabulary steps.

d) *P-CTLE / P-CTLE32: throughput-oriented extension.*: P-CTLE is a hardware-oriented variant of CTLE, not a replacement for CTLE-PSO. It quantizes the activation vector to 16 uniform levels and builds a 256-entry product LUT $L_p[a, w] = L_a[a] \cdot L_w[w]$ once per matvec call (256 multiplications, amortized over $M \times N$ iterations at 36,000:1 for the embedding). The inner loop then executes $acc += L_p[a_idx \| w_idx]$ without any per-element float multiply. On the ESP32-P4, eliminating FMUL.S from the inner loop saves ≈ 4 RISC-V cycles per iteration, yielding **832.5 ± 0.8 ms/tok (1.201 tok/s)**—a **19.6 %** gain over CTLE. The cost is activation quantization error: TinyStories PPL rises from 15.93 to 33.14. Increasing to 32 activation levels (P-CTLE32) reduces this to 20.30 at identical throughput. P-CTLE and P-CTLE32 demonstrate that CTLE-compressed tensors can be mapped to multiplier-free product-LUT datapaths, motivating future FPGA or RISC-V softcore micro-NPU implementations.

e) *Tradeoff summary.*: Four operating points emerge. **CTLE-PSO** gives the best balance of footprint and throughput at 4-bit storage: 0.965 tok/s, 7.61 MB, 21.5 MB PSRAM free. **INT4BW** provides better quality (PPL 10.71/7,120) but is 34 % slower and uses the same 9.51 MB footprint as **CTLE-5b**. **CTLE-5b** combines INT4BW-level quality (PPL 10.00/8,226) with INT4U-class throughput (0.986 tok/s), making it the preferred 5-bit design point. **P-CTLE32** maximizes throughput (1.201 tok/s) for multiplier-free datapaths at 20.30 TS PPL. Rankings are stable across both corpora and all three prompts.

f) *Reproducibility.*: Experiments use a fixed ESP-IDF v6.1 build. Model binaries are generated by `compress.py` (`--method {kmeans,ga,psa,pctle,ctle5}`, `--k {16,32}`, seed 42) and flashed to a SPIFFS partition at

0x310000. The firmware prints a CSV line per prompt that records format, model size, load time, prefill time, generation time, throughput, and heap state; Tables IV and V are derived from these device-side logs.

VI. CONCLUSION

CTLE executes TinyLM transformer layers directly from a learned codebook-plus-index representation, avoiding dense weight reconstruction and scale-based dequantization. The matvec kernel streams packed indices against a register/cache-resident codebook; no intermediate weight buffer is allocated. On ESP32-P4, CTLE-PSO loads a 7.61 MB model and achieves 0.965 tok/s with 21.5 MB PSRAM free. PSO reduces TinyStories PPL by 17.2 % over K-means by better optimizing the 9.2M-entry embedding tensor. INT4BW achieves strong quality (PPL 10.71) but is 34 % slower due to per-group PSRAM scale reads. P-CTLE removes the per-element multiply via a 256-entry product LUT, reaching 1.201 tok/s—a throughput-oriented design point for multiplier-free hardware.

CTLE-5b ($K=32$, 5-bit packed indices, 9.51 MB) has been implemented and benchmarked on the ESP32-P4. It achieves PPL 10.00 on TinyStories—surpassing INT4BW (10.71)—and 1,014.2 ms/tok (0.986 tok/s), 34 % faster than INT4BW, because `decode5_group` introduces no additional PSRAM stream. **P-CTLE32** raises activation quantization to 32 levels (512-entry product LUT), improving TinyStories PPL from 33.14 to 20.30 at identical 832.4 ms/tok throughput, as the LUT build is amortized over 32,000 vocabulary steps.

Limitations: single TinyLM family; reconstruction-error objective; no energy measurement; per-tensor codebooks. Future work includes RVV kernel optimization, hardware power metering, and a streaming product-LUT micro-NPU on FPGA.

ACKNOWLEDGMENTS

This work was supported by ANID BECAS/DOCTORADO NACIONAL 21242003. *AI-tool disclosure (IEEE policy)*: Claude (Anthropic) was used for manuscript editing assistance and code verification. The author is responsible for all technical content, experimental results, and claims.

REFERENCES

- [1] S. Saha, S. Sandha, and M. Srivastava, “Machine learning for microcontroller-class hardware: A review,” *IEEE Sensors Journal*, 2022.
- [2] P.-E. Novac *et al.*, “Quantization and deployment of deep neural networks on microcontrollers,” *Sensors*, 2021.
- [3] A. Elhanashi *et al.*, “Advancements in TinyML: Applications, limitations, and impact on IoT devices,” *Electronics*, 2024.
- [4] A. Rezk *et al.*, “Comparison of LUT-based nonlinearity implementations for recurrent neural networks,” in *ISCAS*, 2020.
- [5] M. Rusci *et al.*, “Mixed-precision inference quantization of speech enhancement with attention-based neural networks,” *Front. Comput. Sci.*, 2022.
- [6] C. Sakr, “Look-up table LSTM model compression,” in *ISCAS*, 2023.
- [7] E. Frantar *et al.*, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *ICLR*, 2023.
- [8] S. Kim *et al.*, “SqueezeLLM: Dense-and-sparse quantization,” in *ICML*, 2024.
- [9] T. Dettmers *et al.*, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *NeurIPS*, 2022.
- [10] J. Lin *et al.*, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *MLSys*, 2024.

- [11] A. Karpathy, “llama2.c,” 2023. [Online]. Available: <https://github.com/karpathy/llama2.c>
- [12] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?” *arXiv:2305.07759*, 2023.
- [13] S. Merity *et al.*, “Pointer sentinel mixture models,” in *ICLR*, 2017.
- [14] S.-Z. Lin *et al.*, “LUTIN: Efficient neural network inference with table lookup,” in *Proc. ACM/IEEE Int. Symp. Low Power Electron. Design (ISLPED)*, 2024, doi: 10.1145/3665314.3670804.